SOEN 201 – Object Oriented Analysis & Design Assignment

Campus Hostel Booking System | Complete Student Guide

Kisii University | School of Computing and Information Technology

Reference materials: Lecturer OOSAD slides (main oosad slides notes.pdf) + SOEN 201 UML Building Blocks notes

1. What the Lecturer Requires (Quick Overview)

You must design **one connected set of UML models** for the Campus Hostel Booking System and give short explanations. All diagrams must use correct UML notation taken from the lecturer's slides (slides 1–233) and the UML Building Blocks notes. Keep every name consistent across diagrams. Label any extra assumption you make. No source code is required. Final diagrams should be drawn in QSEE SUPERLITE, then copied by hand into your exercise book.

Five deliverables must appear in your submission:

1. Five testable requirements + allocation priority rule + three assumptions.
2. Use-case diagram + one fully written use case (Allocate Room).
3. Class diagram with multiplicities + answer on whether Allocation is a class.
4. One sequence diagram (demand exceeds capacity) + one state diagram (Allocation lifecycle).
5. Short consistency check linking sequence messages to class operations and the confirmation deadline.

2. System Scenario (Facts You Must Model)

Area	Facts to model
Halls & rooms	Several halls; single / double / four-person rooms; some halls single-gender, others mixed; each hall has a caretaker.
Applications	Students apply once per semester; may request hall, room type and roommate; applications have opening and closing dates.
Allocation	When demand exceeds capacity the office considers year of study, home distance, previous accommodation, unpaid fees and disciplinary record. Exact ranking rule is unclear – students must confirm offers by a deadline or the room returns to the pool.
After allocation	Students may request transfers or leave early; released rooms may go to a waiting list; rooms under repair cannot be allocated.

3. Deliverable 1 – Requirements

Write five **testable** requirements that each begin with the words "The system shall". A requirement is testable if you can later prove whether it is true or false by looking at system behaviour.

3.1 Five Requirements

R1. The system shall allow a registered student to submit exactly one hostel application per semester before the published closing date.

R2. The system shall record the student's preferred hall, preferred room type and (optionally) preferred roommate when an application is submitted.

R3. The system shall allocate a room only when the chosen room is free, is not under repair, and respects the hall's gender policy.

R4. The system shall issue an offer with a clear confirmation deadline; if the student does not confirm by that deadline the offer shall automatically lapse and the room shall return to the available pool.

R5. The system shall place a student on the waiting list when demand exceeds capacity and shall notify the student when a suitable room becomes available.

3.2 Allocation Priority Rule (Assumption – labelled)

Assumption A1 (Priority Rule): When demand exceeds capacity the system ranks eligible students by the following ordered criteria (highest priority first): (1) higher year of study, (2) greater home distance from campus, (3) student has no previous university accommodation, (4) student has no unpaid fees, (5) student has a clean disciplinary record. Students with unpaid fees or an active disciplinary record are placed at the bottom of the ranking.

3.3 Three Explicit Assumptions

A1 – The priority ordering shown above is used whenever demand exceeds capacity (exact rule was not given by the lecturer).

A2 – A confirmation deadline of 7 calendar days is attached to every offer; the system automatically lapses any unconfirmed offer after that period.

A3 – Only one active application is permitted per student per semester; a new application cannot be submitted until the previous one is withdrawn, allocated or rejected.

4. Deliverable 2 – Use Cases

Draw a use-case diagram that shows the system boundary, the main actors and the use cases they interact with. Then write one use case (Allocate Room) in full detail.

4.1 Use-Case Diagram

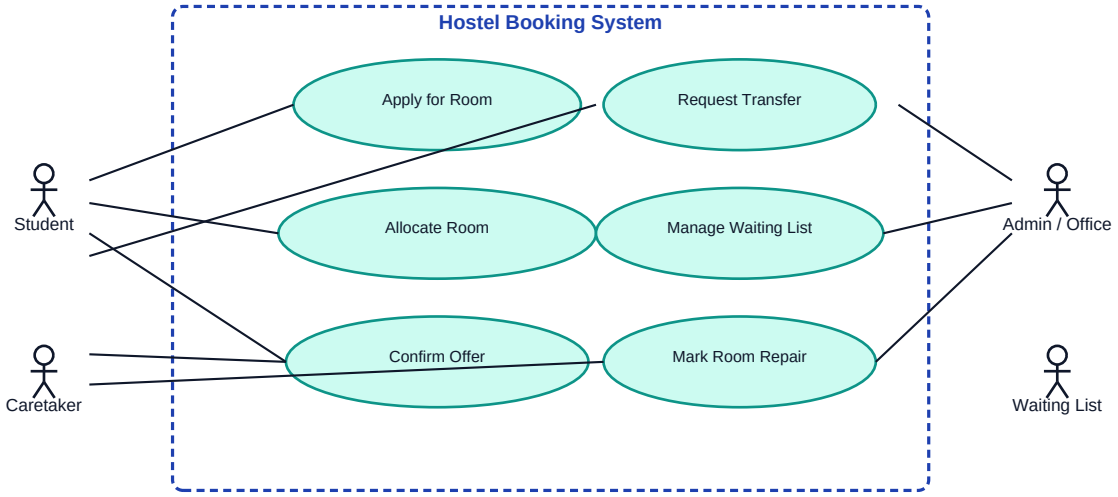

Figure 1 – Use-case diagram for the Campus Hostel Booking System (actors outside the boundary, use cases inside).

4.2 Fully Specified Use Case: Allocate Room

Field	Content
Use case name	Allocate Room
Actor	Admin / Office (primary); Student (secondary – receives notification)
Precondition	Application period has closed; at least one valid application exists; room data is up to date.
Main success scenario	1. Office selects the set of pending applications. 2. System ranks applications using the priority rule (A1). 3. For the highest-ranked student the system finds a free room that matches preference and gender policy. 4. System creates an Allocation in state Offered and sets the confirmation deadline (A2). 5. System notifies the student of the offer and the deadline. 6. Student confirms within the deadline → Allocation moves to Confirmed.
Exception flow E1	Demand exceeds capacity (no free matching room). E1.1 System places the student on the WaitingList. E1.2 System notifies the student that they are on the waiting list. E1.3 Use case ends.
Exception flow E2	Student does not confirm by the deadline. E2.1 System automatically changes Allocation status to Lapsed. E2.2 Room returns to the available pool. E2.3 System may offer the room to the next student on the WaitingList.

5. Deliverable 3 – Class Diagram

Show the main classes, their key attributes, important operations and the association multiplicities. All names must match those used in the use-case and sequence diagrams.

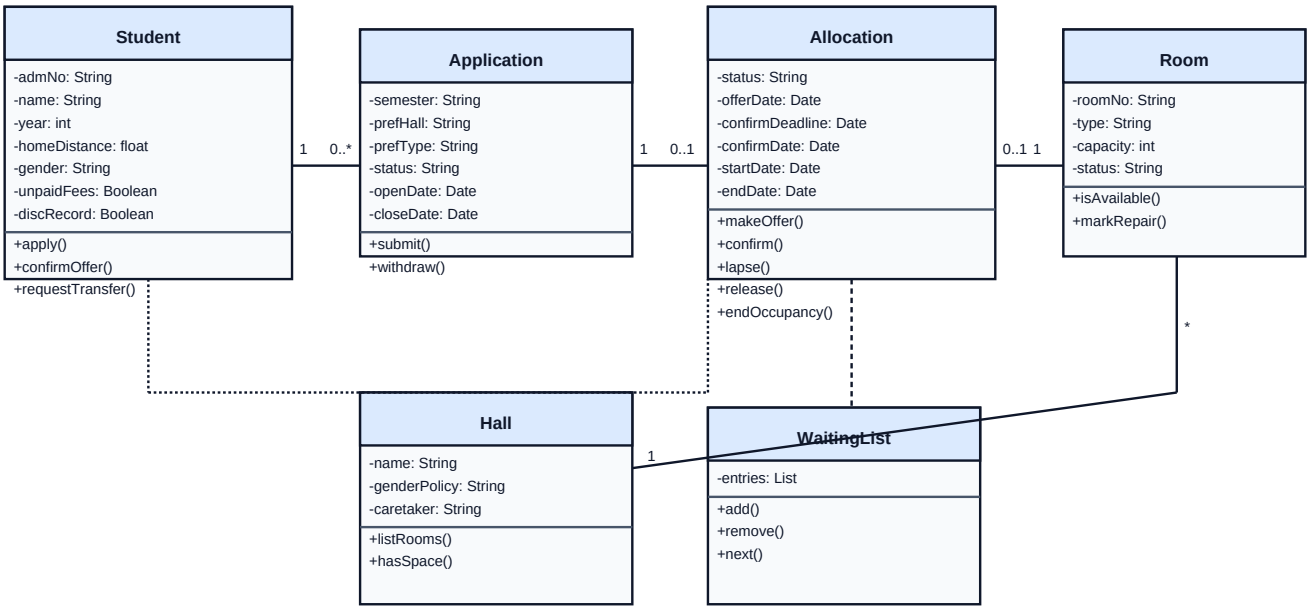

Figure 2 – Class diagram (main classes only). Multiplicities shown on associations.

5.1 Is Allocation a class or an attribute of Student?

Answer: Allocation must be modelled as a **class**, not merely an attribute of Student. Reasons (from UML Building Blocks notes – structural elements):

- An Allocation has its own identity, status, dates and lifecycle (Offered → Confirmed → Occupied / Lapsed / Released).
- It participates in associations with Room, Application and WaitingList; an attribute cannot do this.
- It owns operations (makeOffer, confirm, lapse, release) that change its state – typical of a class, not a simple attribute.

Therefore Allocation is a first-class structural element (see lecturer notes on Class and structural elements).

6. Deliverable 4 – Behaviour Diagrams

6.1 Sequence Diagram – Allocating when demand exceeds capacity

This sequence shows the interaction that occurs when no free room matches the student's request. Messages are numbered and must later be checked against operations in the class diagram.

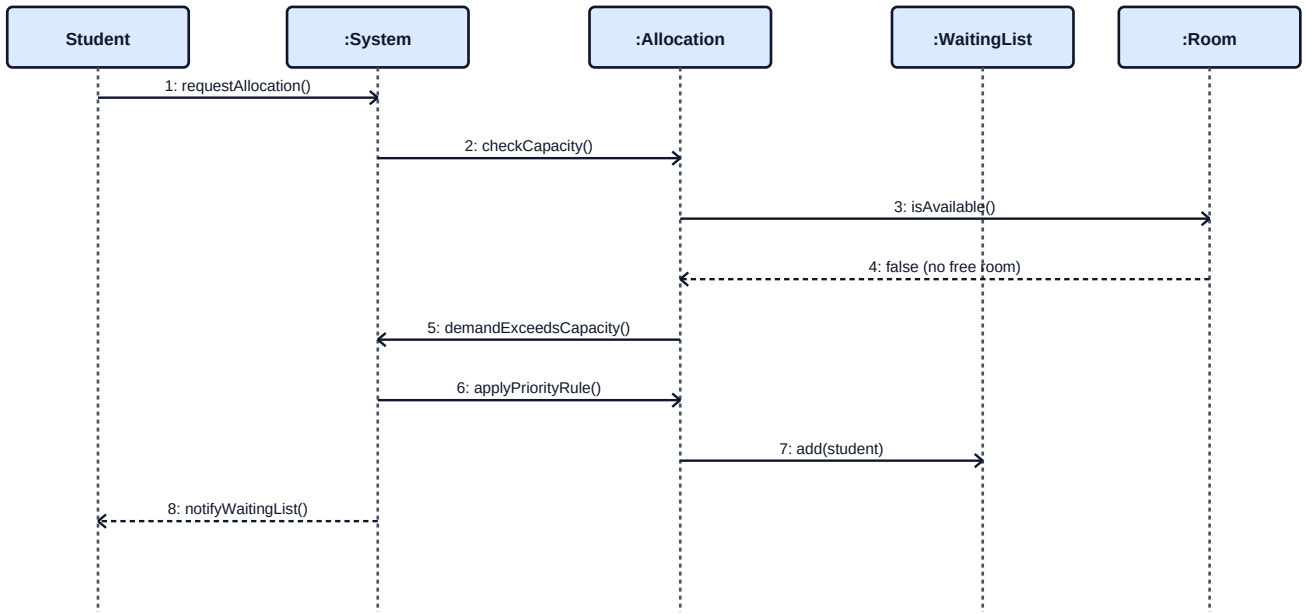

Figure 3 – Sequence diagram: request that ends up on the WaitingList because demand exceeds capacity.

6.2 State Diagram – Allocation lifecycle

The state diagram starts when an offer is created and ends when occupancy finishes or the offer lapses. The confirmation deadline is enforced by the transition from Offered to Lapsed.

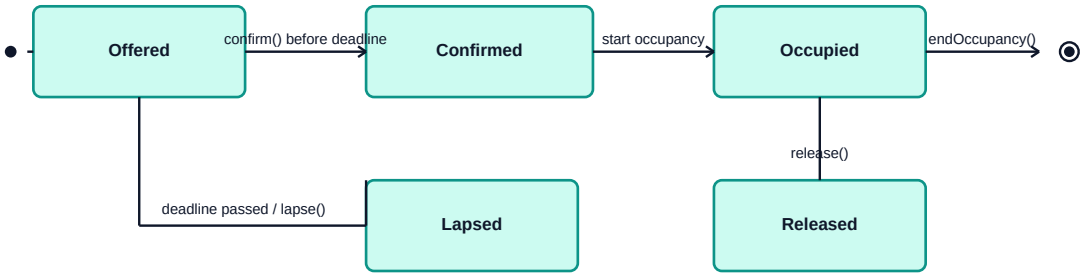

Figure 4 – State diagram for a single Allocation object (offer → confirmation / lapse → occupancy → end).

7. Deliverable 5 – Consistency Check

In a short paragraph explain two things: (a) how the messages in the sequence diagram match operations declared in the class diagram, and (b) how the state diagram enforces the confirmation deadline.

7.1 Sequence messages ↔ Class operations

Every message shown on the sequence diagram corresponds to an operation that appears on one of the classes:

Sequence message	Class that owns the operation
requestAllocation()	System / Student (initiates)
checkCapacity() / isAvailable()	Room.isAvailable()
applyPriorityRule()	Allocation (or System helper)
add(student)	WaitingList.add()
makeOffer() / confirm() / lapse()	Allocation.makeOffer(), confirm(), lapse()

Because the operations already exist on the classes, the sequence diagram is consistent with the class diagram (see lecturer notes on interaction vs structural elements).

7.2 State diagram enforces the confirmation deadline

In the state diagram an Allocation is created in the **Offered** state and is given a confirmDeadline attribute (Assumption A2). Two transitions leave the Offered state:

- **confirm()** – only valid if the current date is still before confirmDeadline → moves to Confirmed.
- **lapse()** – triggered automatically when the deadline passes without confirmation → moves to Lapsed and the room returns to the pool.

Thus the state machine itself guarantees that an offer cannot stay open forever; the deadline is part of the object's behaviour, exactly as required by the scenario.

8. Quick Submission Checklist

- ■ Five "The system shall …" requirements are written and testable.
- ■ Allocation priority rule is stated and labelled as an assumption (A1).
- ■ Three assumptions (A1–A3) are clearly listed.
- ■ Use-case diagram shows system boundary, actors and use cases with correct UML notation.
- ■ Allocate Room use case has actor, precondition, main steps and two exception flows.
- ■ Class diagram contains Student, Application, Allocation, Room, Hall, WaitingList with attributes, operations and multiplicities.
- ■ Explicit answer given: Allocation is a class (not an attribute).
- ■ Sequence diagram for "demand exceeds capacity" path is drawn and messages are numbered.
- ■ State diagram for Allocation shows Offered → Confirmed / Lapsed → Occupied → end.
- ■ Consistency paragraph links sequence messages to class operations and explains the deadline transition.
- ■ All names are identical across every diagram.
- ■ Diagrams prepared in QSEE SUPERLITE then copied by hand; group members' names & admission numbers appear on the form.

Reference: This guide follows the UML building blocks, relationship notations and diagram guidelines contained in the lecturer's OOSAD slides (main oosad slides notes.pdf) and the SOEN 201 UML Building Blocks revision notes. All diagrams use the standard UML notation taught in class (solid/dashed lines, hollow triangles, ellipses for use cases, rounded rectangles for states, etc.).